# Agrupamento (“Clustering”): uma técnica de aprendizado não-supervisionado

**Prof. Aldebaro Klautau**
**Universidade Federal do Pará (UFPA)**

www.lasse.ufpa.br & www.ppgee.ufpa.br

# Unsupervised learning: use cases

Clustering

The goal is to group similar instances together into *clusters*. Clustering is a great tool for data analysis, customer segmentation, recommender systems, search engines, image segmentation, semi-supervised learning, dimensionality reduction, and more.

Anomaly detection

The objective is to learn what "normal" data looks like, and then use that to detect abnormal instances, such as defective items on a production line or a new trend in a time series.

Density estimation

This is the task of estimating the *probability density function* (PDF) of the random process that generated the dataset. Density estimation is commonly used for anomaly detection: instances located in very low-density regions are likely to be anomalies. It is also useful for data analysis and visualization.

# Clustering with K-means

```
from sklearn.cluster import KMeans
k = 5
kmeans = KMeans(n_clusters=k)
y_pred = kmeans.fit_predict(X)
```

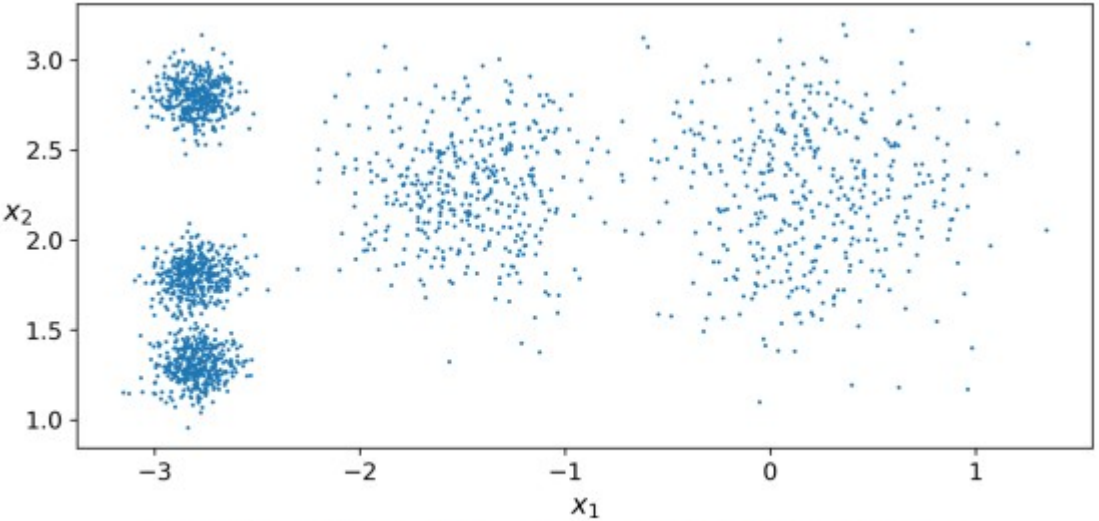


*Figure 9-2. An unlabeled dataset composed of five blobs of instances*

**TIP**

It is important to scale the input features before you run K-Means, or the clusters may be very stretched and K-Means will perform poorly. Scaling the features does not guarantee that all the clusters will be nice and spherical, but it generally improves things.

```
>>> kmeans.cluster_centers_
array([[-2.80389616,  1.80117999],
       [ 0.20876306,  2.25551336],
       [-2.79290307,  2.79641063],
       [-1.46679593,  2.28585348],
       [-2.80037642,  1.30082566]])
```

You can easily assign new instances to the cluster whose centroid is closest:

```
>>> X_new = np.array([[0, 2], [3, 2], [-3, 3], [-3, 2.5]])
>>> kmeans.predict(X_new)
array([1, 1, 2, 2], dtype=int32)
```

If you plot the cluster's decision boundaries, you get a Voronoi tessellatior (see Figure 9-3, where each centroid is represented with an X).

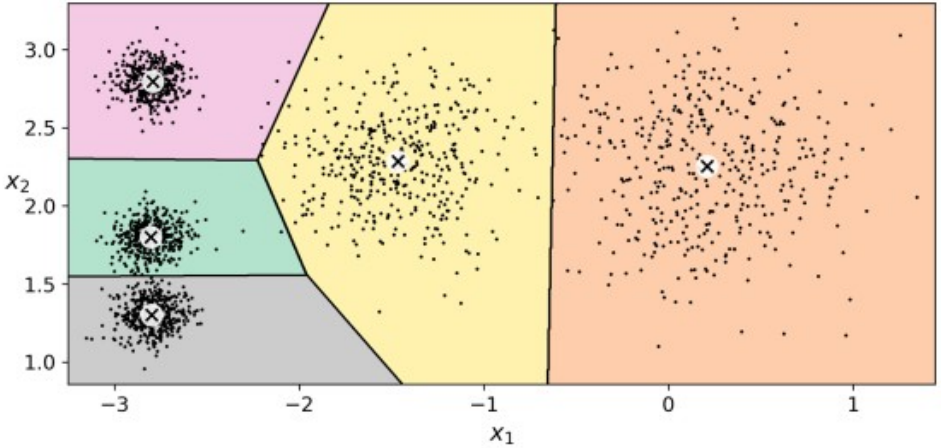


*Figure 9-3. K-Means decision boundaries (Voronoi tessellation)*

# To run K-means

- Need a definition of distance (or “distortion”)
- The mathematical equation to calculate the “centroid”, the vector that minimizes the distances from the points to the centroid
- We also need to define K
- Example:
  - Distance: Euclidean
  - Centroid: Average vector
  - K: Let us say that we want to end up having K=10 vectors (centroids)

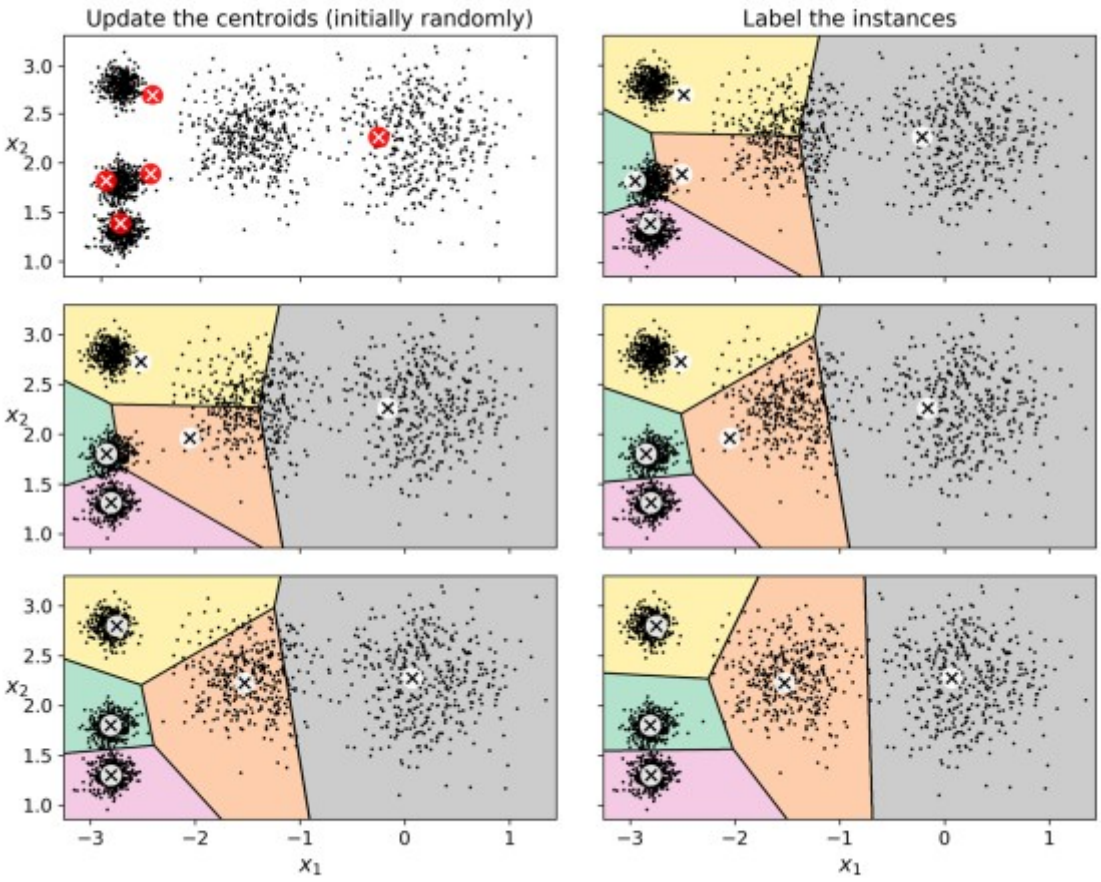


*Figure 9-4. The K-Means algorithm*

Although the algorithm is guaranteed to converge, it may not converge to the right solution (i.e., it may converge to a local optimum): whether it does or not depends on the centroid initialization. Figure 9-5 shows two suboptimal solutions that the algorithm can converge to if you are not lucky with the random initialization step.

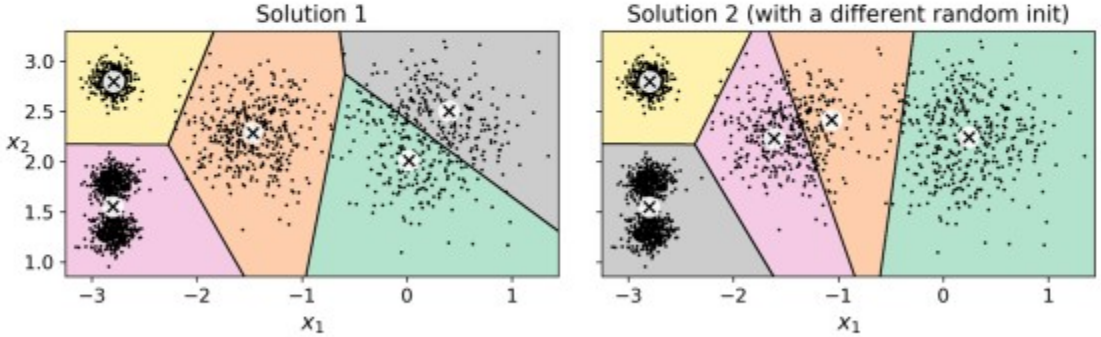


*Figure 9-5. Suboptimal solutions due to unlucky centroid initializations*

## Finding the optimal number of clusters

So far, we have set the number of clusters $k$ to 5 because it was obvious by looking at the data that this was the correct number of clusters. But in general, it will not be so easy to know how to set $k$, and the result might be quite bad if you set it to the wrong value. As you can see in Figure 9-7, setting $k$ to 3 or 8 results in fairly bad models.

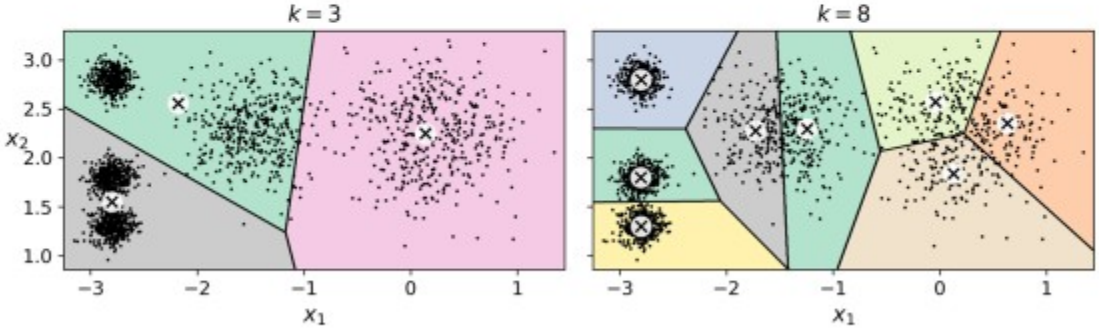


*Figure 9-7. Bad choices for the number of clusters: when k is too small, separate clusters get merged (left), and when k is too large, some clusters get chopped into multiple pieces (right)*

You might be thinking that we could just pick the model with the lowest inertia, right? Unfortunately, it is not that simple. The inertia for $k$=3 is 653.2, which is much higher than for $k$=5 (which was 211.6). But with $k$=8, the inertia is just 119.1. The inertia is not a good performance metric when trying to choose $k$ because it keeps getting lower as we increase $k$. Indeed, the more clusters there are, the closer each instance will be to its closest centroid, and therefore the lower the inertia will be. Let's plot the inertia as a function of $k$ (see Figure 9-8).

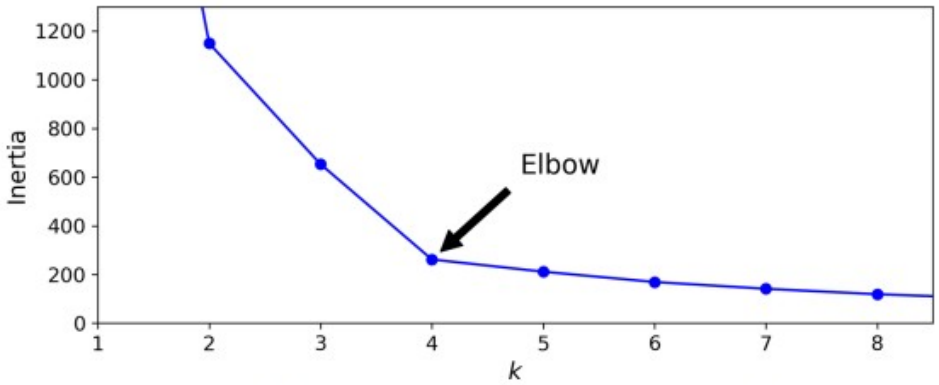


*Figure 9-8. When plotting the inertia as a function of the number of clusters k, the curve often contains an inflexion point called the "elbow"*

As you can see, the inertia drops very quickly as we increase $k$ up to 4, but then it decreases much more slowly as we keep increasing $k$. This curve has roughly the shape of an arm, and there is an "elbow" at $k = 4$. So, if we did not know better, 4 would be a good choice: any lower value would be dramatic, while any higher value would not help much, and we might just be splitting perfectly good clusters in half for no good reason.

To compute the silhouette score, you can use Scikit-Learn's `silhouette_score()` function, giving it all the instances in the dataset and the labels they were assigned:

```
>>> from sklearn.metrics import silhouette_score
>>> silhouette_score(X, kmeans.labels_)
0.655517642572828
```

Let's compare the silhouette scores for different numbers of clusters (see Figure 9-9).

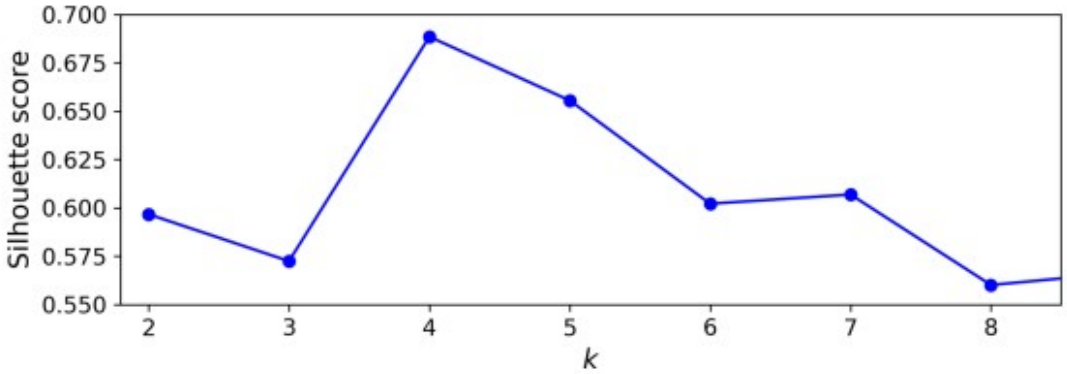


*Figure 9-9. Selecting the number of clusters k using the silhouette score*

As you can see, this visualization is much richer than the previous one: although it confirms that $k = 4$ is a very good choice, it also underlines the fact that $k = 5$ is quite good as well, and much better than $k = 6$ or 7. This was not visible when comparing inertias.

- Métrica "silhouette" é mais adequada que "inertia". Ela foi capaz de indicar para o exemplo, que k=6, 7 ou 8 não são adequados